More Codegen and Arrays

CS 1622

Jarrett Billingsley

Class Announcements

- ?

Quick Recap - Our Calling Convention

Caller

- remember that the **caller** passes all arguments **on the stack.**

```
glob = f(1, 2);
```



```
addi sp, sp, -8
li   t0, 1
sw   t0, 0(sp)
li   t0, 2
sw   t0, 4(sp)
jal  f
sw   v0, glob
```

push the two arguments…

call the function…

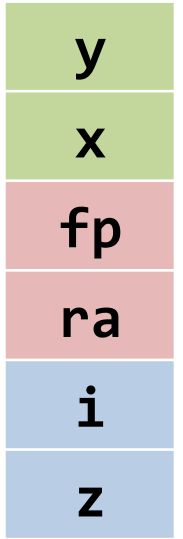
v0 is the return value (if any).

and remember the ATV rule: **after a `jal`, a, t, v registers' values may differ, but s registers' values will be the same.**

Callee

- every function has a **stack frame** to hold the arguments, locals, and saved registers for that call.

```
fn f(x: int, y: int) {
  let z = x + y;
  for i in 0, z {
    println_i(i);
  }
}
```



`4(fp)`

`0(fp)`

arguments are accessed with **positive** offsets from **fp**;

`-12(fp)`

`-16(fp)`

and **locals** are accessed with **negative** offsets from **fp**.

these are the saved copies of the ***caller's*** **fp/ra** registers.

sp always points at the top of the stack.

Prologue and Epilogue

- the stack frame is set up by the **prologue** and torn down by the **epilogue;** these will surround every function.

```
f:
   sw   fp, -4(sp)
   sw   ra, -8(sp)
   move fp, sp
   addi sp, sp, -n

   code goes here!

   lw   ra, -8(fp)
   lw   fp, -4(fp)
   addi sp, sp, x
   jr   ra
```

here, ***n*** represents the size (in bytes) of **all locals and saved registers.**

and here, ***x*** represents the size (in bytes) of **all locals, saved registers, *and arguments.***

this leaves the stack completely clean.

The Code Inside Functions

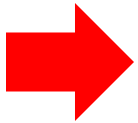
Not a tutorial

- this isn't going to be a point-by-point explanation of how to generate code for every statement and expression.
- rather, I want to give you an *idea* of what generating code is like...
  - and to go over some of the trouble spots that come up.
- so, this section might be a bit rambly and unfocused, and that's why: it's just a bunch of codegen-related ideas.
- after all, project 4 is where you'll be generating code, and I don't wanna give too much away ;)

Block statements

- the body of a function is a **{** block **}**, which is just several statements.
- this is easy to generate code for: you just concatenate the code.

```
{
    stmt1;
    stmt2;
    stmt3;
}
```



```
code for stmt1
code for stmt2
code for stmt3
```

this works because the **runtime semantics** of block statements are a very good match for how code is naturally executed by the CPU.

but that's not the case for all statements...

Accessing variables

- HLLs **hide** how variables are accessed at the point of use.

```
let g = 10;
fn main() {

  let x = 1;

  g = 2;

  print_i(x);

  print_i(g);
}
```

here we **store** to a **local** variable:

```
sw t0, -12(fp)
```

here we **store** to a **global** variable:

```
sw t0, g
```

here we **load** from a **local** variable:

```
lw t0, -12(fp)
```

here we **load** from a **global** variable:

```
lw t0, g
```

Evaluating expressions

- things start to get really tricky here.

```
g = x + y;
```



```
lw   t0, 0(fp)
lw   t1, 4(fp)
add  t0, t0, t1
sw   t0, g
```

now we have to come up with **registers** to use for each part of the expression.

the order we do things in can affect how many registers we need.

```
g = x + y + z;
```



```
lw   t0, 0(fp)
lw   t1, 4(fp)
add  t0, t0, t1
lw   t1, 8(fp)
add  t0, t0, t1
sw   t0, g
```

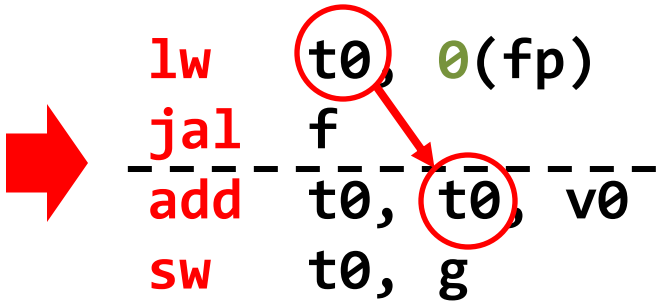
or?

```
lw   t0, 0(fp)
lw   t1, 4(fp)
lw   t2, 8(fp)
add  t0, t0, t1
add  t0, t0, t2
sw   t0, g
```

The need for saved registers

- once you mix function calls in, it gets all kinds of complicated.

```
g = x + f();
```



this is bad code! we are violating the ATV rule.

this is because the value of **`t0`** from before the call may not survive until its use after the call. ***`f`*** *is allowed to change* ***`t0`***.

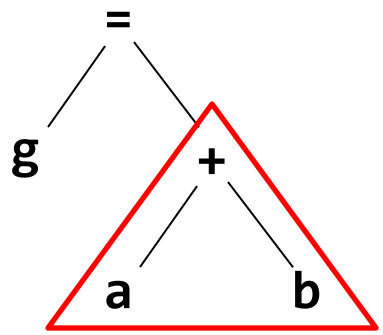
so we have to use the **saved** (**`s`**) registers for some temporaries, so their values will persist across calls.

to make things simpler, we could use *only* the saved registers, and ignore the **`t`** registers altogether.

A little detour

- expressions are represented as **trees**, and trees ♡ stacks, so…
- let's first imagine evaluating an AST by pushing and popping things **on a stack,** instead of dealing with registers.

```
g = a + b;
```

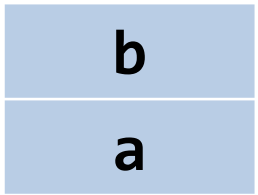


let's evaluate the assignment's rhs.

1. push a



2. push b



3. pop them, push sum

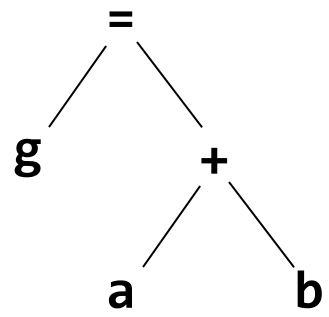


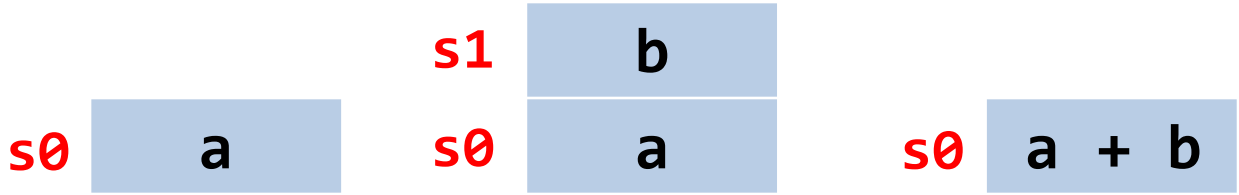
you can use this strategy to evaluate any expression of any complexity.
(let's do a few examples.)

Ok? And? How does that help?

- well… we can map that *imaginary* stack **onto the registers.**

```
g = a + b;
```





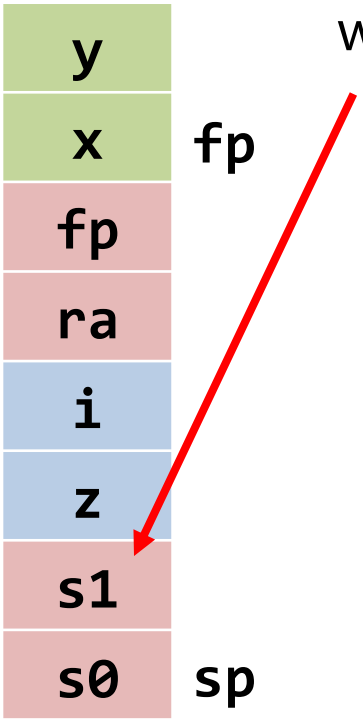
this tells us **what register to put each value into!**

```
lw   s0, 0(fp) # a
lw   s1, 4(fp) # b
add  s0, s0, s1
sw   s0, g
```

but now we come to an important issue: to be allowed to use the saved registers, **we must save the caller's versions of them.**

The last piece of the calling convention

- only *after* we have generated the code for a function, do we know **how many saved registers it needs.**
- so there is one more part of the stack frame:



we have to save the caller's values of **any saved registers this function uses.**

in the prologue, we will have to store each of these saved registers to the stack…

…and in the epilogue, we will load them back.

this is how the saved register contract works.
I taught it in 447 and it's just as relevant now.

Some last register allocation thoughts

- can we use **`t`** registers at all?
  - sure, **as long as we can prove that there are no function calls** from the time we *put* a value in it to the time we *use* that value.
  - that *might* be pretty simple: look at the AST "to the right" and if there are no “function call” nodes, we can use a **`t`** register instead.
  - this is more complicated, but it will use **fewer** (or *no*) **s registers,** meaning **less time/space** spent saving/restoring them.
- there are only 8 saved registers (**`s0`** .. **`s7`**). **what if we need more?**
  - uhh 
  - well, this register allocation scheme will work for simple code.
  - we'll talk about a **much more sophisticated** algorithm at the end of the course, which will solve this problem (and others, like using **a** registers for arguments instead of the stack).

Control flow statements

- assuming we've got expressions worked out, another surprisingly tricky bit is **control flow statements:** conditionals and loops.
  - specifically, **the conditions** for these statements can be hard to generate *good* code for. (but that's not a primary goal here.)

```
if i < 10 {
   ...
}
```



```
 lw   s0, -16(fp)
 bge  s0, 10, _end
 ...
_end:
```

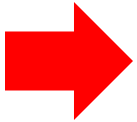
getting the condition that simple is hard to do; we might do something like this instead:

```
 lw   s0, -16(fp)
 li   s1, 10
 slt  s0, s0, s1
 beq  s0, zero, _end
 ...
_end:
```

Lazy operators ("short-circuiting")

- recall that like in most languages, logical **and** and **or** will **only evaluate their rhs if they have to** – making their codegen *very* different from the other operators.

```
if i < 10 or i > 20 {
   ...
}
```



```
   lw   s0, -16(fp)
   li   s1, 10
   slt  s0, s0, s1
   bne  s0, zero, _true
   lw   s0, -16(fp)
   li   s1, 20
   sgt  s0, s0, s1
   beq  s0, zero, _end
_true:
   ...
_end:
```

see how it skips right to the code when the first condition is true?

You can assign comparison operators into variables

- it seems like no one teaches you this but this is totally valid code in Java (and in lots of other languages too):

```
boolean b = x < 10;
boolean d = i < 10 || i > 20;
```

- because of this, you may not be able to use the same codegen algorithm for e.g. logical OR in all cases!
  - you still have to do the short-circuiting…
  - but now you're producing a value to be put into a variable
  - which, depending on how your codegen works, could be a very different thing from "writing a condition for a loop"

Arrays

Adding features is Fun™

- okay. now let's assume we've implemented lexing, parsing, semantic analysis, and codegen for **all the basic features** of our language:
  - variables, functions, ints, bools, strings. just the simple stuff.
- let's now think about how we'd go about adding a **new feature** to the language, **from design all the way through to codegen.**
  - and along the way, this might give you an appreciation for how parts of the language interact with one another.
- we'll be adding **arrays,** which will work very similarly to how they do in Java, but with some slightly different syntax.

Minimum Viable Product

- a good place to start is by writing some code to get a feel for how we want this feature to **look** and **work.**
- let's focus on the absolute **core features of arrays:** allocating them, accessing their items, and getting their length.

```
let a = new [int; 10];
```

this syntax might look a bit odd, but it's meant to echo a similar syntax used in Rust. it's also *easier to parse* than `new int[10]`.

```
a[0] = 5;
```

I think it's clear what's happening here.

```
for i in 0, a.length { println_i(a[i]); }
```

and this is familiar too!

Lexing and Parsing

- for lexing, we only need to add two tokens to our lexer: **[** and **]**.
- for parsing, we've got a few new rules to add.

```
new [int; 10]
```

```
New:        'new' (StructNew | ArrayNew)
StructNew:  Id '(' ')'
ArrayCtor:  '[' Type ';' Exp ']'
```

```
a[0] = 5;
```

```
PostfixOp:  FuncCall | Field | Index
Index:      '[' Exp ']'
```

one more important rule: we have to be able to write **array types,** for things like function arguments and struct fields.

```
fn f(a: [int])
```

```
Type:       VoidType | 'bool' | 'int' |
            'string' | FuncType |
            StructType | ArrayType
ArrayType:  '[' Type ']'
```

Semantic analysis: typing rules

- arrays don't interact with the *naming* rules at all, so let's skip to types.

`new [T; n]`
- `n` must be `int`
- the type of the whole expression is `[T]`

`a[i]`
- `a` must be `[T]`
- `i` must be `int`
- the type of the whole expression is `T`

`a.length`
- if `a` is `[T]`:
  - the only allowed field is `length`
  - the type of the whole expression is `int`
  - and this **cannot be the lhs of =**

More semantics: evaluation rules

- these explain **what the code does,** which can also require **runtime error checking.**

`new [T; n]`

- dynamically allocates an array on the heap.
- `n` must be `>= 0`.
  - if not, the program halts with an error.

`a[i]`

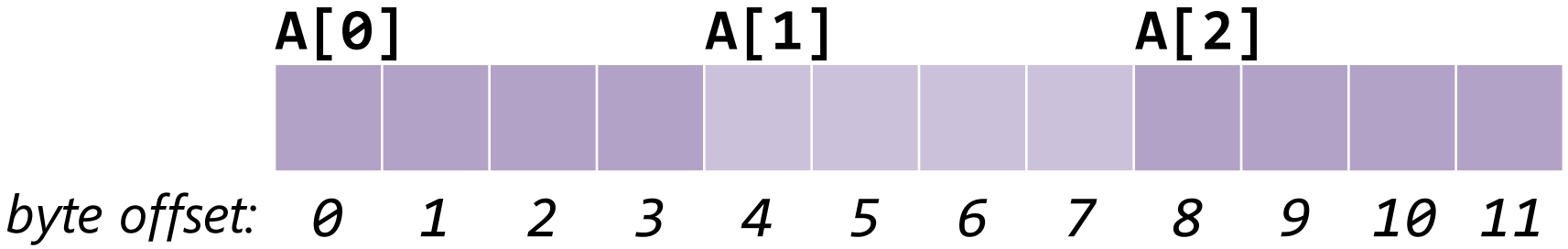
- gets the `i`th item in `a`, zero-based.
- `a` is evaluated before `i`.
- `i` must be `>= 0` and `< a.length`.
  - if not, the program halts with an error.

`a.length`

- gets the length of `a`.

Runtime representation: data

- as you hopefully learned in 447, an array is represented in memory by placing its items next to each other, **equidistantly spaced.**
- consider an array of 3 `int`s which are **32 bits** (**4 Bytes**) each:



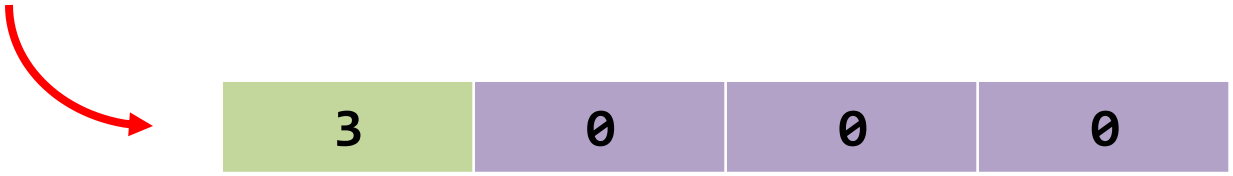
given a pointer A that points to the **beginning** of the array, we can get the ***i***th item with the formula **A + S·i,** where **S** is the size of one item.

here, **S = 4**, so **A + 4·i** gives us the byte offset of item **i**.

Runtime representation: length

- each array **also has to remember its length,** because there's nothing to tell you where "the end of the array" is.

```
let a = new [int; 3];
```



the **length** comes first and takes up 4 bytes.

so now our item access formula must be offset by 4 to skip the length: **A + S·i + 4**

(different types use different values of S, so we can't do **A + S(i + 1)** in general.)

now we have all we need to be able to implement **`a[i]`** and **`a.length`**! but what about **`new[int; 3]`**?

Runtime library: allocating arrays

- **dynamic memory allocation** is something the runtime library is responsible for. so, we'll need a function to allocate arrays.
- `fn rt$new_array(item_size: int, len: int)`
  - this will allocate an array of **`len`** items, where each item is **`item_size`** bytes; zero out the data; set its **`length`** field to **`len`**; and return a pointer to it.
  - it will also crash the program if **len** is negative, as per the evaluation semantics.
- the code generation will turn array allocations into calls to this function, using a constant for **`item_size`** based on the type.
  - e.g. for an array of **bool**, **`item_size`** is 1; for **int** it's 4; etc.

Codegen

- finally, code generation is fairly straightforward.

`new [T; n]`	generated as a call to `rt$new_array(S, n)`
`a.length`	load an int from the memory `a` points to
`a[i]`	first, crash if `i` is negative or `>= a.length`. if not, calculate address as `a + Si + 4`; then load/store

and we're done! yay, our language has arrays now!

Or does it?

- this is just the *minimum viable version* of this feature.
- what about things like:
  - **array initializers/literals,** for allocating and filling in arrays at the same time like in Java: `int[] a = new int[]{ 1, 2, 3 };`
  - **better multi-dimensional array initialization,** again like Java: `int[][] a = new int[10][20];`
    - remember, this is an *array of arrays:* **each item** in the outer array (the `10` dimension) is a reference to an array of 20 ints
  - **standard library functions** for sorting, searching, filling, modifying arrays?
- such a simple-seeming feature, and yet so many things to consider…
  - but think about how each of these might be implemented **on top of** the basic features we talked about here.